Note de séance SQL n°7

DISTINCT, BETWEEN, NULL, NULLIF, CAST, UNION, self join | 2026-10-22

1. Les notions

Notion	Règle à retenir
DISTINCT	Supprime les lignes identiques, sur la combinaison de toutes les colonnes affichées.
COUNT(DISTINCT x)	Compte les valeurs différentes, hors NULL. Indispensable pour compter les éléments du côté « un » après
BETWEEN a AND b	Intervalle, bornes incluses. Pour des horodatages, préférer >= et <.
IS NULL / IS NOT NULL	On ne compare jamais avec = NULL. NULL n'est pas une chaîne vide : prévoir les deux.
NULLIF(a, b)	Renvoie NULL si a = b. Sert à traiter les chaînes vides comme des NULL, et à éviter une division par zéro
CAST(x AS type)	Convertit (SIGNED, DECIMAL, DATE, CHAR). Traiter le vide avant de convertir : CAST(NULLIF(x, '') AS
UNION / UNION ALL	Empile des résultats : même nombre de colonnes, types compatibles, noms de la première requête. UNIO
Self join	Une table jointe avec elle-même, avec deux alias. Hiérarchies (manager) et paires (a.id < b.id).

2. Préparation des données

```
SET SQL_SAFE_UPDATES = 0;

ALTER TABLE clients ADD COLUMN telephone VARCHAR(20);
UPDATE clients SET telephone = '0812345678' WHERE id = 1;
UPDATE clients SET telephone = '0823456789' WHERE id = 2;
UPDATE clients SET telephone = '0834567890' WHERE id = 4;
UPDATE clients SET telephone = '0845678901' WHERE id = 5;
UPDATE clients SET telephone = '' WHERE id = 7;     -- chaîne vide (clients 3, 6, 8 : NULL)

CREATE TABLE import_ventes (id INT, produit VARCHAR(50), quantite VARCHAR(10), prix_total VARCHAR(
INSERT INTO import_ventes VALUES (1, 'Telephone', '2', '500');
INSERT INTO import_ventes VALUES (2, 'Casque', '4', '160');
INSERT INTO import_ventes VALUES (3, 'Souris', '0', '0');
INSERT INTO import_ventes VALUES (4, 'Clavier', '3', '90');
INSERT INTO import_ventes VALUES (5, 'Tablette', '', '300');

CREATE TABLE anciens_clients (id INT, prenom VARCHAR(50), ville VARCHAR(50));
INSERT INTO anciens_clients VALUES (101, 'Awa', 'Kinshasa');
INSERT INTO anciens_clients VALUES (102, 'Blaise', 'Kisangani');
INSERT INTO anciens_clients VALUES (103, 'Claire', 'Goma');
INSERT INTO anciens_clients VALUES (104, 'Jean', 'Lubumbashi');

CREATE TABLE equipe (id INT, prenom VARCHAR(50), poste VARCHAR(50), manager_id INT);
INSERT INTO equipe VALUES (1, 'Esther', 'Directrice', NULL);
INSERT INTO equipe VALUES (2, 'Paul', 'Chef analyste', 1);
INSERT INTO equipe VALUES (3, 'Nadine', 'Analyste', 2);
INSERT INTO equipe VALUES (4, 'Kevin', 'Analyste', 2);
INSERT INTO equipe VALUES (5, 'Rachel', 'Stagiaire', 3);
```

Partie 1 : DISTINCT et COUNT(DISTINCT)

1.1 : Villes où habitent des clients, sans doublon

```
SELECT DISTINCT ville FROM clients;
```

Résultat : Kinshasa, Lubumbashi, Goma

1.2 : Catégories commandées par des clients de Kinshasa

```
SELECT DISTINCT p.categorie
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN produits p ON p.id = o.produit_id
WHERE c.ville = 'Kinshasa';
```

Résultat : Electronique, Accessoires (la table villes est inutile ici)

1.3 : Clients différents ayant commandé : (a) dans commandes, (b) avec jointure

```
SELECT COUNT(DISTINCT client_id) AS nb_clients FROM commandes;

SELECT COUNT(DISTINCT o.client_id) AS nb_clients
FROM commandes o
INNER JOIN clients c ON c.id = o.client_id;
```

Résultat : 5 puis 4 : le client 9 (commande orpheline) n'existe pas dans clients

1.4 : Par province : clients, clients acheteurs, commandes

```
SELECT v.province,
       COUNT(DISTINCT c.id) AS nb_clients,
       COUNT(DISTINCT o.client_id) AS nb_clients_acheteurs,
       COUNT(o.id) AS nb_commandes
FROM villes v
LEFT JOIN clients c ON c.ville = v.ville
LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY v.province;
```

Résultat : Haut-Katanga 2, 1, 1 ; Kinshasa 4, 3, 5 ; Nord-Kivu 2, 0, 0 (partir de villes)

1.5 : Catégories différentes achetées par client

```
SELECT c.id, c.prenom, COUNT(DISTINCT p.categorie) AS nb_categories
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
LEFT JOIN produits p ON p.id = o.produit_id
GROUP BY c.id, c.prenom;
```

Résultat : Awa 2, Jean 1, Marie 1, Grace 1, les 4 autres 0

1.6 : Le 4.9 avec jointures et DISTINCT

```
SELECT DISTINCT p.nom, p.categorie
FROM produits p
INNER JOIN commandes o ON o.produit_id = p.id
INNER JOIN clients c ON c.id = o.client_id
WHERE c.ville = 'Kinshasa';
```

Résultat : Telephone, Casque, Souris, Clavier, Tablette

Partie 2 : BETWEEN, IS NULL, NULLIF, CAST

2.1 : Montants entre 100 et 300 (inclus)

```
SELECT id, produit, montant FROM commandes
WHERE montant BETWEEN 100 AND 300;
```

Résultat : Telephone, Tablette, Montre

2.2 : Commandes entre le 1er février et le 31 mars 2026

```
SELECT id, produit, date_commande FROM commandes
WHERE date_commande BETWEEN '2026-02-01' AND '2026-03-31';
```

Résultat : Ordinateur, Souris, Clavier, Tablette

2.3 : Clients sans téléphone : (a) NULL, (b) NULL ou vide

```
SELECT prenom FROM clients WHERE telephone IS NULL;

SELECT prenom FROM clients WHERE telephone IS NULL OR telephone = '';
```

Résultat : (a) Marie, David, Eric ; (b) plus Sarah

2.4 : Téléphone, 'inconnu' si NULL ou vide

```
SELECT prenom, COALESCE(NULLIF(telephone, ''), 'inconnu') AS tel FROM clients;
```

Résultat : 4 numéros, 4 inconnu

2.5 : Clients avec et sans numéro (CASE WHEN)

```
SELECT CASE WHEN telephone IS NULL OR telephone = '' THEN 'Sans numéro'
            ELSE 'Avec numéro' END AS statut,
       COUNT(*) AS nb_clients
FROM clients
GROUP BY statut;
```

Résultat : Avec numéro 4, Sans numéro 4

2.6 : Prix unitaire, NULL si quantité 0 ou vide

```
SELECT produit,
       CAST(prix_total AS SIGNED) / NULLIF(CAST(NULLIF(quantite, '') AS SIGNED), 0) AS prix_unitai
FROM import_ventes;
```

Résultat : Telephone 250, Casque 40, Souris NULL, Clavier 30, Tablette NULL

2.7 : Tri texte et tri numérique

```
SELECT * FROM import_ventes ORDER BY prix_total;

SELECT * FROM import_ventes ORDER BY CAST(prix_total AS SIGNED);
```

Résultat : Texte : 0, 160, 300, 500, 90 ; nombre : 0, 90, 160, 300, 500

Partie 3 : UNION, UNION ALL, self join

3.1 : Prénoms et villes de clients et anciens_clients, sans doublon

```
SELECT prenom, ville FROM clients
UNION
SELECT prenom, ville FROM anciens_clients;
```

Résultat : 10 lignes

3.2 : Même liste en gardant tout

```
SELECT prenom, ville FROM clients
UNION ALL
SELECT prenom, ville FROM anciens_clients;
```

Résultat : 12 lignes : Awa (Kinshasa) et Jean (Lubumbashi) apparaissent deux fois

3.3 : Villes de trois tables, triées

```
SELECT ville FROM clients
UNION
SELECT ville FROM anciens_clients
UNION
SELECT ville FROM villes
ORDER BY ville;
```

Résultat : Goma, Kinshasa, Kisangani, Lubumbashi

3.4 : Membre, poste et manager

```
SELECT e.prenom AS membre, e.poste, COALESCE(m.prenom, 'Aucun') AS manager
FROM equipe e
LEFT JOIN equipe m ON e.manager_id = m.id;
```

Résultat : Esther Aucun, Paul Esther, Nadine Paul, Kevin Paul, Rachel Nadine

3.5 : Collaborateurs directs de chacun

```
SELECT m.id, m.prenom, COUNT(e.id) AS nb_collaborateurs
FROM equipe m
```

```
LEFT JOIN equipe e ON e.manager_id = m.id
GROUP BY m.id, m.prenom;
```

Résultat : Esther 1, Paul 2, Nadine 1, Kevin 0, Rachel 0

3.6 : Paires de clients de la même ville, une seule fois

```
SELECT a.prenom AS client_1, b.prenom AS client_2, a.ville
FROM clients a
INNER JOIN clients b ON a.ville = b.ville AND a.id < b.id;
```

Résultat : 8 paires : 6 à Kinshasa, 1 à Lubumbashi, 1 à Goma

3.7 : Clients plus jeunes de la même ville

```
SELECT c.id, c.prenom, COUNT(d.id) AS nb_plus_jeunes
FROM clients c
LEFT JOIN clients d ON d.ville = c.ville AND d.age < c.age
GROUP BY c.id, c.prenom;
```

Résultat : Awa 1, Marie 0, Grace 2, Eric 3, Jean 0, Sarah 1, Patrick 1, David 0

3. Pièges rencontrés

• **COUNT(DISTINCT)** : sans DISTINCT, un COUNT(c.id) après jointure compte chaque client une fois par commande (Kinshasa : 6 au lieu de 4).

• **Partir de la bonne table** : « pour chaque province de la table villes » impose FROM villes.

• **Tables inutiles** : la ville est déjà dans clients, la jointure avec villes n'apporte rien.

• **UNION compare des lignes entières** : omettre la ville fusionnerait deux personnes de même prénom.

• **NULLIF avant CAST** : comparer un nombre à '' ne marche que grâce à une conversion implicite de MySQL.

• **COUNT(condition)** : COUNT compte les valeurs non NULL, y compris les 0. Pour compter sous condition : COUNT(CASE WHEN ... THEN 1 END).

• **Condition dans le ON** : dans un self join avec LEFT JOIN, la condition de comparaison (d.age < c.age) va dans le ON, pas dans le WHERE.

• **Liste de sortie** : écrire en commentaire ce que la requête doit afficher (ville, poste, colonnes demandées) avant de l'écrire.